

尚硅谷大数据技术之 Spark 入门

(作者：尚硅谷研究院)

版本：V5.0

第 1 章 Spark 概述

1.1 什么是 Spark

回顾：Hadoop 主要解决，海量数据的存储和海量数据的分析计算。

Spark 是一种基于内存的快速、通用、可扩展的大数据分析计算引擎。

1.2 Hadoop 与 Spark 历史

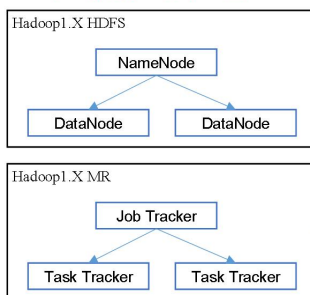
回顾Hadoop历史

1、Hadoop历史

2011年发布1.x版本

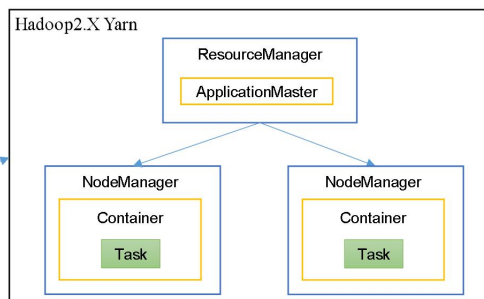
2012年发布稳定版本

2013年10月发布2.x版本（增加YARN）



Hadoop2.x 解决了Hadoop1.x中的问题

- 1) NameNode高可用
- 2) 将资源调度和任务调度和解耦
- 3) 计算框架可插拔



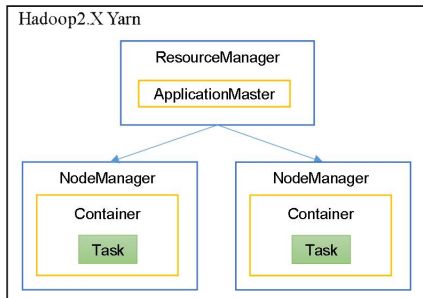
Hadoop1.x 问题

- 1) NameNode不能高可用
- 2) MR框架将资源调度和任务调度耦合在一起
- 3) MR框架基于磁盘计算，性能比较低

让天下没有难学的技术

1、Hadoop 历史

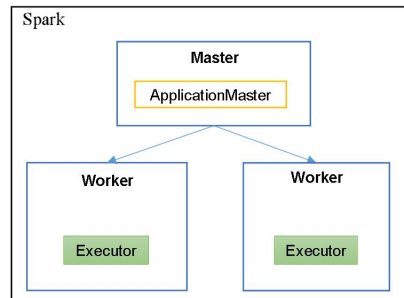
2011年发布1.x版本
2012年发布稳定版本
2013年10月发布2.x版本（增加YARN）



Yarn是为了其他框架也可以使用，所以用了Container，兼容性更好

2、Spark 历史

2009年诞生于加州大学伯克利分校AMPLab，项目采用Scala编写。
2010年开源；
2013年6月成为Apache孵化项目（基于Hadoop问题，将资源和任务调度分开）
2014年2月成为Apache顶级项目



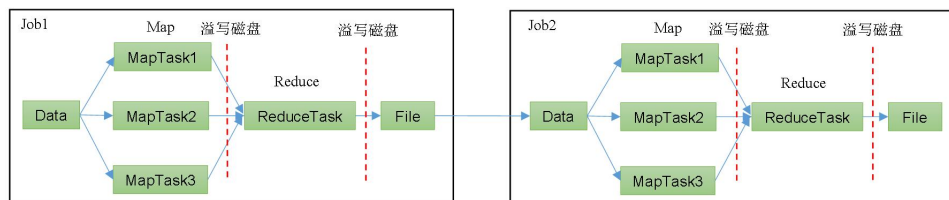
Spark的资源调度框架就是为了自己使用，所以没有必要用Container，效率更高 *让天下没有难学的技术*

Hadoop 的 Yarn 框架比 Spark 框架诞生的晚，所以 Spark 自己也设计了一套资源调度框架。

1.3 Hadoop 与 Spark 框架对比

1、Hadoop MR框架

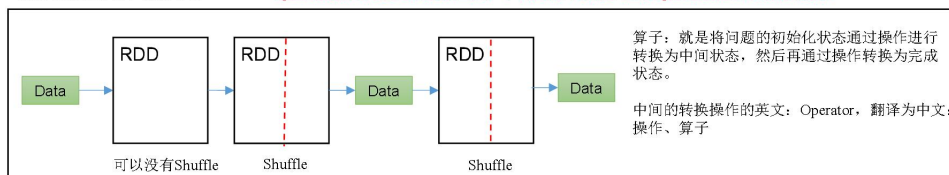
从数据源获取数据，经过分析计算后，将结果输出到指定位置，核心是一次计算，不适合迭代计算。



2、Spark 框架

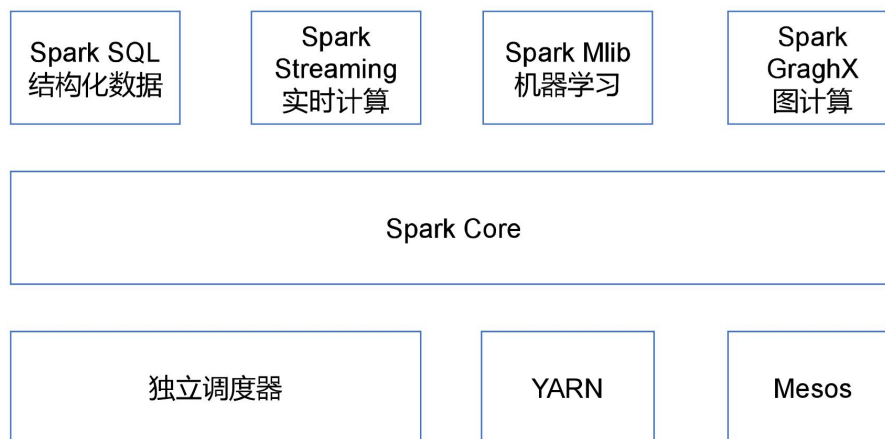
支持迭代式计算，图形计算

Spark框架计算比MR快的原因是：中间结果不落盘。注意Spark的Shuffle也是落盘的。



1.4 Spark 内置模块

Spark 内置模块



让天下没有难学的技术

Spark Core: 实现了 Spark 的基本功能，包含任务调度、内存管理、错误恢复、与存储系统交互等模块。Spark Core 中还包含了对**弹性分布式数据集**(Resilient Distributed DataSet, 简称 RDD)的 API 定义。

Spark SQL: 是 Spark 用来操作结构化数据的程序包。通过 Spark SQL, 我们可以使用 SQL 或者 Apache Hive 版本的 HQL 来查询数据。Spark SQL 支持多种数据源, 比如 Hive 表、Parquet 以及 JSON 等。

Spark Streaming: 是 Spark 提供的对实时数据进行流式计算的组件。提供了用来操作数据流的 API, 并且与 Spark Core 中的 RDD API 高度对应。

Spark MLlib: 提供常见的机器学习功能的程序库。包括分类、回归、聚类、协同过滤等, 还提供了模型评估、数据 导入等额外的支持功能。

Spark GraphX: 主要用于图形并行计算和图挖掘系统的组件。

集群管理器: Spark 设计为可以高效地在一个计算节点到数千个计算节点之间伸缩计算。为了实现这样的要求, 同时获得最大灵活性, Spark 支持在各种集群管理器(Cluster Manager)上运行, 包括 Hadoop YARN、Apache Mesos, 以及 Spark 自带的一个简易调度器, 叫作**独立调度器**。

更多 Java -大数据 -前端 -python 人工智能资料下载, 可百度访问: [尚硅谷官网](http://www.shangguigu.com)

Spark 得到了众多大数据公司的支持,这些公司包括 Hortonworks、IBM、Intel、Cloudera、MapR、Pivotal、百度、阿里、腾讯、京东、携程、优酷土豆。当前百度的 Spark 已应用于大搜索、直达号、百度大数据等业务;阿里利用 GraphX 构建了大规模的图计算和图挖掘系统,实现了很多生产系统的推荐算法;腾讯 Spark 集群达到 8000 台的规模,是当前已知的世界上最大的 Spark 集群。

1.5 Spark 特点

Spark特点



- 1) 快:** 与Hadoop的MapReduce相比, Spark基于内存的运算要快100倍以上, 基于硬盘的运算也要快10倍以上。Spark实现了高效的DAG执行引擎, 可以通过基于内存来高效处理数据流。计算的中间结果是存在于内存中的。
- 2) 易用:** Spark支持Java、Python和Scala的API, 还支持超过80种高级算法, 使用户可以快速构建不同的应用。而且Spark支持交互式的Python和Scala的Shell, 可以非常方便地在这些Shell中使用Spark集群来验证解决问题的方法。
- 3) 通用:** Spark提供了统一的解决方案。Spark可以用于, 交互式查询 (Spark SQL)、实时流处理 (Spark Streaming)、机器学习 (Spark MLlib) 和图计算 (GraphX)。这些不同类型的处理都可以在同一个应用中无缝使用。减少了开发和维护的人力成本和部署平台的物力成本。
- 4) 兼容性:** Spark可以非常方便地与其他开源产品进行融合。比如, Spark可以使用Hadoop的YARN和Apache Mesos作为它的资源管理和调度器, 并且可以处理所有Hadoop支持的数据, 包括HDFS、HBase等。这对于已经部署Hadoop集群的用户特别重要, 因为不需要做任何数据迁移就可以使用Spark的强大处理能力。

让天下没有难学的技术

第 2 章 Spark 运行模式

部署 Spark 集群大体上分为两种模式: **单机模式与集群模式**

大多数分布式框架都支持单机模式, 方便开发者调试框架的运行环境。但是在生产环境中, 并不会使用单机模式。因此, 后续直接按照集群模式部署 Spark 集群。

下面详细列举了 Spark 目前支持的部署模式。

- (1) Local 模式:** 在本地部署单个 Spark 服务
- (2) Standalone 模式:** Spark 自带的任务调度模式。(国内不常用)
- (3) YARN 模式:** Spark 使用 Hadoop 的 YARN 组件进行资源与任务调度。(国内最常用)
- (4) Mesos 模式:** Spark 使用 Mesos 平台进行资源与任务的调度。(国内很少用)

更多 Java -大数据 -前端 -python 人工智能资料下载, 可百度访问: 尚硅谷官网

2.1 Spark 安装地址

- 1) 官网地址: <http://spark.apache.org/>
- 2) 文档查看地址: <https://spark.apache.org/docs/3.3.1/>
- 3) 下载地址: <https://spark.apache.org/downloads.html>
<https://archive.apache.org/dist/spark/>

2.2 Local 模式

Local 模式就是运行在一台计算机上的模式, 通常就是用于在本机上练手和测试。

2.2.1 安装使用

- 1) 上传并解压 Spark 安装包

```
[atguigu@hadoop102 software]$ tar -zxvf spark-3.3.1-bin-hadoop3.tgz -C /opt/module/  
[atguigu@hadoop102 module]$ mv spark-3.3.1-bin-hadoop3 spark-local
```

- 2) 官方求 PI 案例

```
[atguigu@hadoop102 spark-local]$ bin/spark-submit \  
--class org.apache.spark.examples.SparkPi \  
--master local[2] \  
./examples/jars/spark-examples_2.12-3.3.1.jar \  
10
```

可以查看 spark-submit 所有参数:

```
[atguigu@hadoop102 spark-local]$ bin/spark-submit
```

- --class: 表示要执行程序的主类;
- --master local[2]

(1) local: 没有指定线程数, 则所有计算都运行在一个线程当中, 没有任何并行计算

(2) local[K]: 指定使用 K 个 Core 来运行计算, 比如 local[2] 就是运行 2 个 Core 来执行

```
20/09/20 09:30:53 INFO TaskSetManager:  
20/09/15 10:15:00 INFO Executor: Running task 1.0 in stage 0.0 (TID 1)  
20/09/15 10:15:00 INFO Executor: Running task 0.0 in stage 0.0 (TID 0)
```

(3) local[*]: 默认模式。自动帮你按照 CPU 最多核来设置线程数。比如 CPU 有 8 核,

Spark 帮你自动设置 8 个线程计算。

```
20/09/20 09:30:53 INFO TaskSetManager:  
20/09/15 10:15:58 INFO Executor: Running task 1.0 in stage 0.0 (TID 1)  
20/09/15 10:15:58 INFO Executor: Running task 0.0 in stage 0.0 (TID 0)  
20/09/15 10:15:58 INFO Executor: Running task 2.0 in stage 0.0 (TID 2)  
20/09/15 10:15:58 INFO Executor: Running task 4.0 in stage 0.0 (TID 4)  
20/09/15 10:15:58 INFO Executor: Running task 3.0 in stage 0.0 (TID 3)
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

2.3.1 安装使用

1) 解压 spark

```
[atguigu@hadoop102 software]$ tar -zxvf spark-3.3.1-bin-hadoop3.tgz -C /opt/module/
```

2) 进入到/opt/module 目录，修改 spark-3.3.1-bin-hadoop3 名称为 spark-yarn

```
[atguigu@hadoop102 module]$ mv spark-3.3.1-bin-hadoop3/ spark-yarn
```

3) 修改 hadoop 配置文件/opt/module/hadoop/etc/hadoop/yarn-site.xml，添加如下内容

因为测试环境虚拟机内存较少，防止执行过程进行被意外杀死，做如下配置

```
[atguigu@hadoop102 hadoop]$ vim yarn-site.xml
<!--是否启动一个线程检查每个任务正使用的物理内存量，如果任务超出分配值，则直接将其杀掉，默认是 true -->
<property>
  <name>yarn.nodemanager.pmem-check-enabled</name>
  <value>>false</value>
</property>

<!--是否启动一个线程检查每个任务正使用的虚拟内存量，如果任务超出分配值，则直接将其杀掉，默认是 true -->
<property>
  <name>yarn.nodemanager.vmem-check-enabled</name>
  <value>>false</value>
</property>
```

4) 分发配置文件

```
[atguigu@hadoop102 conf]$ xsync /opt/module/hadoop/etc/hadoop/yarn-site.xml
```

5) 修改/opt/module/spark-yarn/conf/spark-env.sh，添加 YARN_CONF_DIR 配置，保证后

续运行任务的路径都变成集群路径

```
[atguigu@hadoop102 conf]$ mv spark-env.sh.template spark-env.sh
[atguigu@hadoop102 conf]$ vim spark-env.sh
```

```
YARN_CONF_DIR=/opt/module/hadoop-3.3.4/etc/hadoop
```

6) 启动 HDFS 以及 YARN 集群

```
[atguigu@hadoop102 hadoop-3.3.1]$ sbin/start-dfs.sh
[atguigu@hadoop103 hadoop-3.3.1]$ sbin/start-yarn.sh
```

7) 执行一个程序

```
[atguigu@hadoop102 spark-yarn]$ bin/spark-submit \
--class org.apache.spark.examples.SparkPi \
--master yarn \
./examples/jars/spark-examples_2.12-3.3.1.jar \
10
```

参数：--master yarn，表示 Yarn 方式运行；--deploy-mode 表示客户端方式运行程序

8) 查看 hadoop103:8088 页面，点击 History，查看历史页面

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

思考：目前是 Hadoop 的作业运行日志展示，如果想获取 Spark 的作业运行日志，怎么办？

ID	User	Name	Application Type	Queue	StartTime	FinishTime	State	FinalStatus	Progress	Tracking URL
application_1576807140637_0003	atguigu	Spark Pi	SPARK	default	Fri Dec 20 10:02:25 +0800 2019	Fri Dec 20 10:02:44 +0800 2019	FINISHED	SUCCEEDED		History

2.3.2 配置历史服务

由于是重新解压的 Spark 压缩文件，所以需要针对 Yarn 模式，再次配置一下历史服务器。

1) 修改 spark-default.conf.template 名称

```
[atguigu@hadoop102 conf]$ mv spark-defaults.conf.template spark-defaults.conf
```

2) 修改 spark-default.conf 文件，配置日志存储路径（写）

```
[atguigu@hadoop102 conf]$ vim spark-defaults.conf
spark.eventLog.enabled      true
spark.eventLog.dir          hdfs://hadoop102:8020/directory
```

3) 修改 spark-env.sh 文件，添加如下配置：

```
[atguigu@hadoop102 conf]$ vim spark-env.sh

export SPARK_HISTORY_OPTS="
-Dspark.history.ui.port=18080
-Dspark.history.fs.logDirectory=hdfs://hadoop102:8020/directory
-Dspark.history.retainedApplications=30"
```

参数 1 含义：WEBUI 访问的端口号为 18080

参数 2 含义：指定历史服务器日志存储路径（读）

参数 3 含义：指定保存 Application 历史记录个数，如果超过这个值，旧的应用程序信息将被删除，这个是内存中的应用数，而不是页面上显示的应用数。

2.3.3 配置查看历史日志

为了能从 Yarn 上关联到 Spark 历史服务器，需要配置 spark 历史服务器关联路径。

目的：点击 yarn(8088)上 spark 任务的 history 按钮，进入的是 spark 历史服务器(18080)，而不再是 yarn 历史服务器（19888）。

1) 修改配置文件/opt/module/spark-yarn/conf/spark-defaults.conf

添加如下内容：

```
spark.yarn.historyServer.address=hadoop102:18080
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网


```
spark.history.ui.port=18080
```

2) 重启 Spark 历史服务

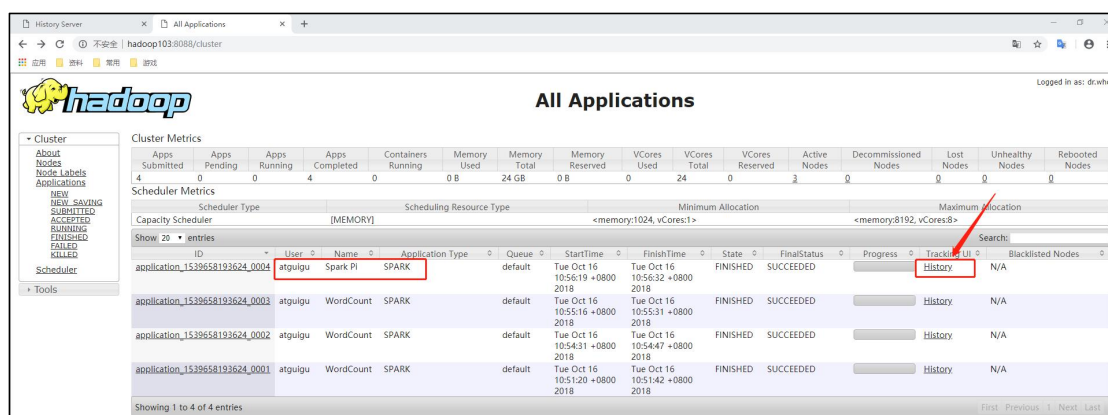
```
[atguigu@hadoop102 spark-yarn]$ sbin/stop-history-server.sh
```

```
[atguigu@hadoop102 spark-yarn]$ sbin/start-history-server.sh
```

3) 提交任务到 Yarn 执行

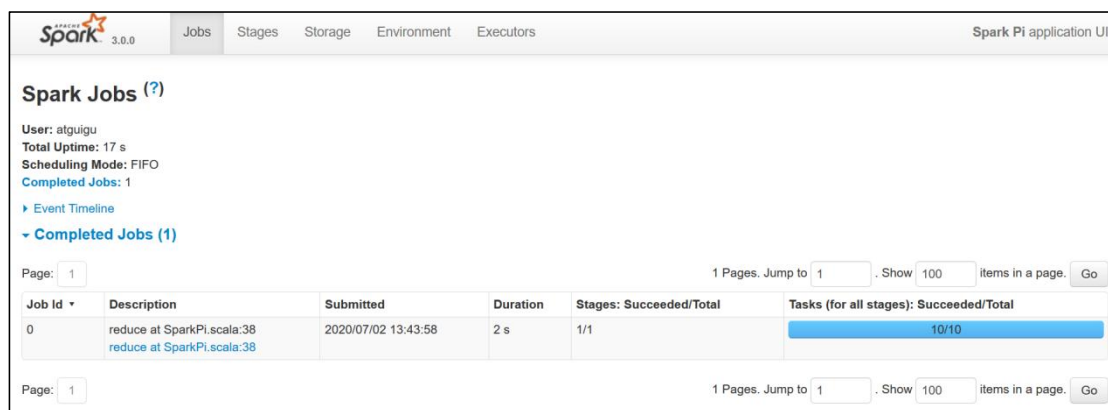
```
[atguigu@hadoop102 spark-yarn]$ bin/spark-submit \
--class org.apache.spark.examples.SparkPi \
--master yarn \
./examples/jars/spark-examples_2.12-3.3.1.jar \
10
```

4) Web 页面查看日志: <http://hadoop103:8088/cluster>



The screenshot shows the Hadoop All Applications web interface. The 'Cluster Metrics' section displays various statistics. The 'Scheduler Metrics' section shows the Scheduler Type as [MEMORY]. The 'Capacity Scheduler' section shows the Scheduling Resource Type as <memory:1024, vCores:1>. The 'Show 20 entries' table lists applications. The first application, 'application_1539658193624_0004', is highlighted. The 'Tracking UI' column for this application has a red box around the 'History' link, which is pointed to by a red arrow.

点击“history”跳转到 <http://hadoop102:18080/>



The screenshot shows the Spark Pi application UI. The 'Spark Jobs (?)' section displays the User as atguigu, Total Uptime as 17 s, and Scheduling Mode as FIFO. The 'Completed Jobs: 1' section shows a table with one job. The job details are as follows:

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	reduce at SparkPi.scala:38 reduce at SparkPi.scala:38	2020/07/02 13:43:58	2 s	1/1	10/10

2.3.4 运行流程

Spark 有 yarn-client 和 yarn-cluster 两种模式，主要区别在于：Driver 程序的运行节点。

yarn-client: Driver 程序运行在客户端，适用于交互、调试，希望立即看到 app 的输出。

yarn-cluster: Driver 程序运行在由 ResourceManager 启动的 APPMaster，适用于生产环境。

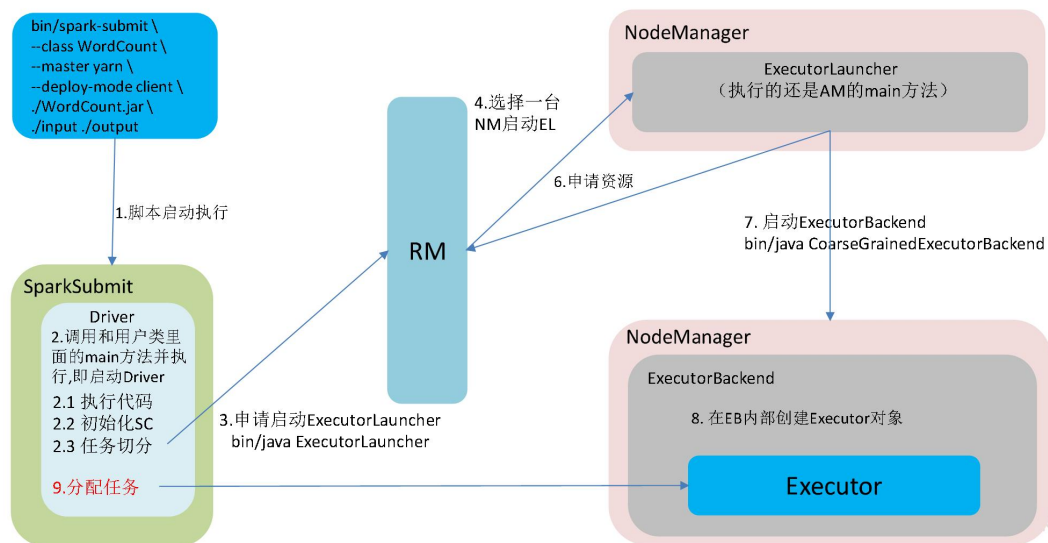
更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

1) 客户端模式（默认）

```
[atguigu@hadoop102 spark-yarn]$ bin/spark-submit \
--class org.apache.spark.examples.SparkPi \
--master yarn \
--deploy-mode client \
./examples/jars/spark-examples_2.12-3.3.1.jar \
10
```



YarnClient 运行模式介绍



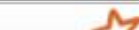
2) 集群模式

```
[atguigu@hadoop102 spark-yarn]$ bin/spark-submit \
--class org.apache.spark.examples.SparkPi \
--master yarn \
--deploy-mode cluster \
./examples/jars/spark-examples_2.12-3.3.1.jar \
10
```

(1) 查看 <http://hadoop103:8088/cluster> 页面，点击 History 按钮，跳转到历史详情页面

ID	User	Name	Application Type	Queue	StartTime	FinishTime	State	FinalStatus	Progress	Tracking UI
application_1576809977138_0004	atguigu	org.apache.spark.examples.SparkPi	SPARK	default	Fri Dec 20	Fri Dec 20 10:58:37	FINISHED	SUCCEEDED		History

(2) <http://hadoop102:18080> 点击 Executors->点击 driver 中的 stdout



3.0.0

Jobs

Stages

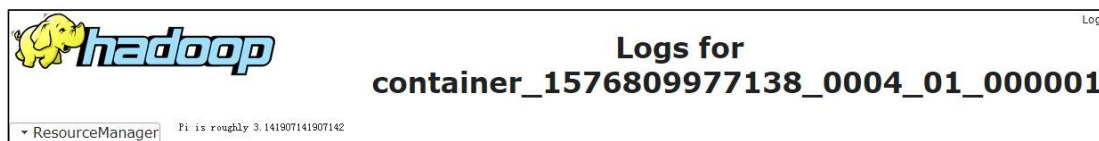
Storage

Environment

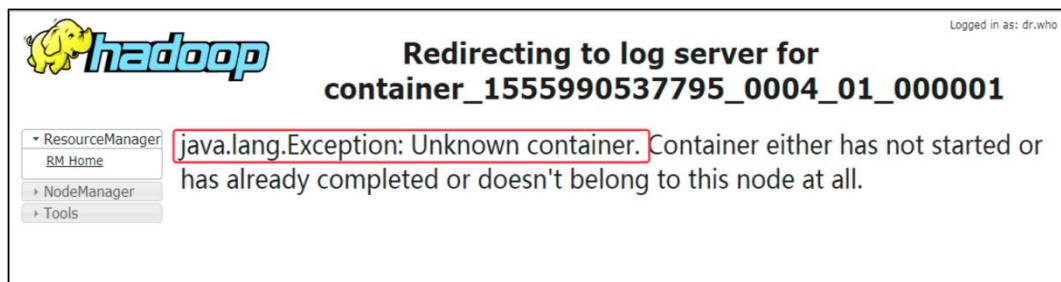
Executors

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Shuffle Input	Shuffle Read	Shuffle Write	Logs
driver	192.168.1.104:60940	Active	0	0.0 B / 384.1 MB	0.0 B	0	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网



注意：如果在 yarn 日志端无法查看到具体的日志，则在 *yarn-site.xml* 中添加如下配置并启动 Yarn 历史服务器

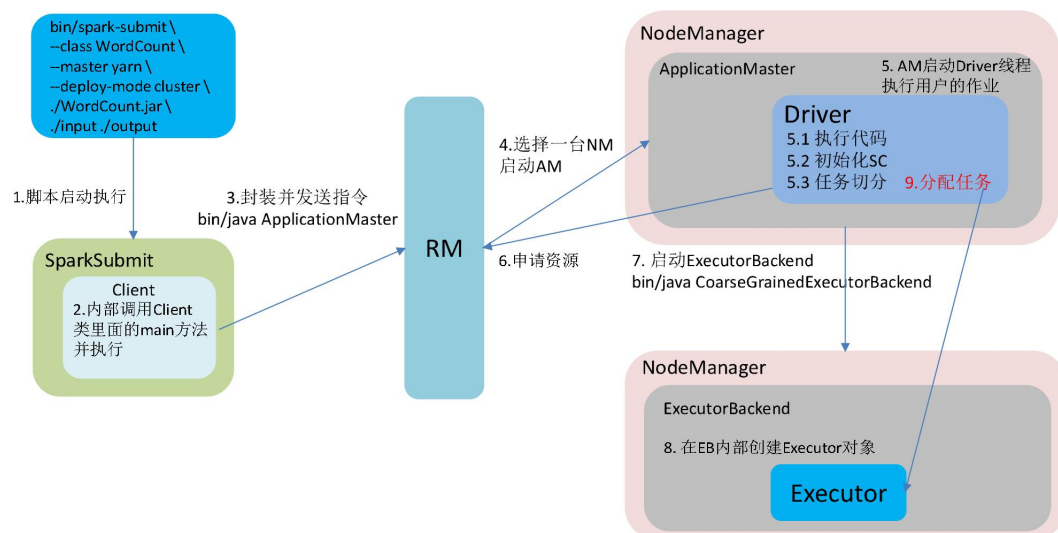


```
<property>
  <name>yarn.log.server.url</name>
  <value>http://hadoop102:19888/jobhistory/logs</value>
</property>
```

注意：hadoop 历史服务器也要启动 `mr-jobhistory-daemon.sh start historyserver`



YarnCluster模式



2.4 Standalone 模式（了解）

Standalone 模式是 Spark 自带的资源调度引擎,构建一个由 **Master + Worker** 构成的 Spark 集群, Spark 运行在集群中。

这个要和 Hadoop 中的 Standalone 区别开来。这里的 Standalone 是指只用 Spark 来搭建一个集群，不需要借助 Hadoop 的 Yarn 和 Mesos 等其他框架。

2.5 Mesos 模式（了解）

Spark 客户端直接连接 Mesos；不需要额外构建 Spark 集群。国内应用比较少，更多的是运用 Yarn 调度。

2.6 几种模式对比

模式	Spark 安装机器数	需启动的进程	所属者
Local	1	无	Spark
Standalone	3	Master 及 Worker	Spark
Yarn	1	Yarn 及 HDFS	Hadoop

2.7 端口号总结

- 1) Spark 查看当前 Spark-shell 运行任务情况端口号：4040
- 2) Spark 历史服务器端口号：18080 （类比于 Hadoop 历史服务器端口号：19888）